

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 8

Processes Talking Through Pipes, in xv6

Course: Operating Systems (Track B: build your own small OS (xv6))

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **1**
1.1 Laboratory Objectives and Scope 1
1.2 Internal xv6 Pipe Architecture 1

2 Part 1: Ping-Pong Communication (`pingpong.c`) **3**
2.1 Implementation Details 3
2.2 Build and Execution Trace 3
2.3 Answers to Questions (Observe – Part 1) 4

3 Part 2: Concurrent Prime Sieve (`primes.c`) **5**
3.1 Design and Algorithm Architecture 5
3.2 Implementation of `user/primes.c` 5
3.3 Pipeline Architecture Diagram 6
3.4 Execution Trace 6
3.5 Answers to Questions (Observe – Part 2) 7

4 Part 3: Stretch Assignment – Implementing `xargs` (`xargs.c`) **9**
4.1 Specification and Design 9
4.2 Implementation of `user/xargs.c` 9
4.3 Execution Trace and Verification 10

5 Conclusions **11**

1 Theoretical Background and Objectives

1.1 Laboratory Objectives and Scope

Inside xv6 there is no glibc, no `strace`, and no manual pages: only the 22 system calls provided by the kernel compiled from source. The objective of Laboratory Work Nr. 8 is to utilize fundamental process and pipe primitives from the lecture to build concurrent UNIX user-space programs:

- **Process and Pipe System Calls**: Directly invoking the core xv6 system calls declared in `user/user.h`:

  `fork()`, `exit()`, `wait()`, `pipe()`, `read()`, `write()`, `close()`, `getpid()`, `printf()`

- **Bidirectional IPC Coordination (`pingpong`)**: Exchanging a byte back and forth between parent and child processes using two distinct anonymous pipes without race conditions or self-reading.

- **Concurrent Prime Sieve (`primes`)**: Implementing Doug McIlroy's concurrent prime sieve pipeline, chaining dynamically created filter stages connected by anonymous pipes.

- **Systems Programming Traps**:

  1. *File descriptor exhaustion*: Adhering to per-process descriptor limits (`NOFILE = 16` in `kernel/param.h`) and system-wide file table limits (`NFILE = 100`) by immediately closing unused and inherited pipe ends.
  2. *Infinite pipe hanging*: Ensuring that all open write descriptors are properly closed once data transmission finishes so downstream filters observe `read() == 0` (EOF).
  3. *Compiler recursion checks*: Satisfying xv6's strict build flags (`-Werror`) by declaring recursive stages with `__attribute__((noreturn))`.

- **Argument Passing (`xargs` – Stretch Goal)**: Building `user/xargs.c` to consume newline-separated inputs and execute target programs with composite argument vectors using `fork()`, `exec()`, and `wait()`.

1.2 Internal xv6 Pipe Architecture

In xv6 (`kernel/pipe.c`), an anonymous pipe is implemented as a bounded in-memory FIFO ring buffer:

```
struct pipe {
  struct spinlock lock;
  char data[PIPESIZE]; // 512 bytes
  uint nread;          // number of bytes read
  uint nwrite;         // number of bytes written
  int readopen;        // read fd is still open
  int writeopen;       // write fd is still open
};
```

- **Synchronization**: When `read()` finds the buffer empty (`nread == nwrite`) and the write end remains open (`writeopen != 0`), the kernel calls `sleep(&pi->nread, &pi->lock)` to suspend the reader. When a writer issues `write()`, it appends data and calls `wakeup(&pi->nread)`, moving sleeping readers to the `RUNNABLE` state.
- **EOF Generation**: When a process calls `close()` on the write end of a pipe, `pipeclose()` sets `writeopen = 0` and wakes any sleeping readers. If the buffer is empty and `writeopen == 0`, `piperead()` immediately returns `0`, signaling End-Of-File.

2 Part 1: Ping-Pong Communication (`pingpong.c`)

2.1 Implementation Details

We implemented `user/pingpong.c`. The parent sends a byte to the child through pipe `p2c`. The child reads it, prints its own PID, sends the byte back through pipe `c2p`, and terminates. The parent reads from `c2p`, prints its own PID, and reaps the child:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];
  char b = 'x';

  if(pipe(p2c) < 0 || pipe(c2p) < 0){
    fprintf(2, "pingpong: pipe failed\n");
    exit(1);
  }
  if(fork() == 0){
    close(p2c[1]); /* child does not write to p2c */
    close(c2p[0]); /* child does not read from c2p */
    if(read(p2c[0], &b, 1) != 1)
      exit(1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); /* parent does not read from p2c */
  close(c2p[1]); /* parent does not write to c2p */
  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  close(p2c[1]);
  close(c2p[0]);
  wait(0);
  exit(0);
}
```

2.2 Build and Execution Trace

We registered `$U/_pingpong\` in the `Makefile` and booted xv6 under QEMU:

```
$ pingpong
4: received ping
3: received pong
```

2.3 Answers to Questions (Observe – Part 1)

(1) Why do you need TWO pipes, and not one shared by both directions?

Pipes in UNIX and xv6 are strictly **unidirectional** byte streams backed by a single kernel ring buffer. If only one shared pipe were used:

- When the parent writes the byte `'x'` and immediately calls `read()`, the parent would race against the child and would almost certainly read back its own byte before the child could execute.
- Sharing a single pipe creates data races, non-deterministic self-consumption of bytes, and deadlock conditions.

Two independent pipes (`p2c` for parent-to-child and `c2p` for child-to-parent) provide isolated channels with dedicated buffers, ensuring deterministic sequencing.

(2) Nothing in your code says "wait until the child has replied". What makes the parent wait?

The parent is suspended by the blocking behavior of the **`read()`** system call. When the parent calls:

```
read(c2p[0], &b, 1);
```

the child has not yet written into `c2p[1]`. In `kernel/pipe.c`, because the buffer is empty and `writeopen` is non-zero, the kernel puts the parent to sleep:

```
while(pi->nread == pi->nwrite && pi->writeopen){
  if(killed(myproc())){ ... }
  sleep(&pi->nread, &pi->lock);
}
```

The parent yields the CPU until the child writes the byte, which triggers `wakeup(&pi->nread)` and allows the parent to resume execution.

(3) Why does the child print a larger PID than the parent?

In xv6, process identifiers (PIDs) are allocated sequentially from a global counter in `kernel/proc.c`:

```
int
allocpid() {
  int pid;
  acquire(&pid_lock);
  pid = nextpid;
  nextpid = nextpid + 1;
  release(&pid_lock);
  return pid;
}
```

The parent process already exists (having received PID 3 when launched by `sh`). When the parent subsequently executes `fork()`, the kernel allocates the next integer in sequence (PID 4) to the child. Therefore, the child always has a strictly higher PID than its parent.

3 Part 2: Concurrent Prime Sieve (`primes.c`)

3.1 Design and Algorithm Architecture

Doug McIlroy's concurrent prime sieve pipeline decomposes prime number discovery into streaming pipeline stages:

1. **Generator Process**: Feeds candidate integers $2, 3, \ldots, 35$ into an initial pipe.
2. **Stage Processes**: Each stage reads its first integer p. Because all multiples of smaller primes have been filtered by earlier stages, p is guaranteed to be prime. The stage prints `"prime p"`.
3. **Filtering and Forwarding**: The stage forks a child stage and passes through only those subsequent numbers n that satisfy $n \pmod{p} \neq 0$. Multiples of p are dropped.
4. **Cascade Termination**: When the input pipe reaches EOF (`read() == 0`), the stage closes its outgoing pipe, waits for its child to exit, and terminates cleanly.

3.2 Implementation of `user/primes.c`

The function `stage()` is annotated with `__attribute__((noreturn))` to satisfy `-Werror=infinite-re`

```c
#include "kernel/types.h"
#include "user/user.h"

#define LIMIT 35

void stage(int) __attribute__((noreturn));

void
stage(int in)
{
  int p, n, fd[2], pid;

  if(read(in, &p, sizeof(p)) != sizeof(p)){
    close(in);
    exit(0);
  }
  printf("prime %d\n", p);

  if(pipe(fd) < 0){
    printf("primes: pipe failed after prime %d\n", p);
    close(in);
    exit(1);
  }
  pid = fork();
  if(pid < 0){
    printf("primes: fork failed after prime %d\n", p);
    close(fd[0]); close(fd[1]); close(in);
    exit(1);
  }
  if(pid == 0){
    close(fd[1]); /* Child reads from fd[0] */
    close(in);    /* Child does not need parent's input pipe */
    stage(fd[0]);
```

```
  }
  close(fd[0]);   /* Parent writes to fd[1] */
  while(read(in, &n, sizeof(n)) == sizeof(n)){
    if(n % p != 0)
      write(fd[1], &n, sizeof(n));
  }
  close(in);
  close(fd[1]);   /* Closes write end to deliver EOF to next stage */
  wait(0);
  exit(0);
}

int
main(int argc, char *argv[])
{
  int fd[2];

  if(pipe(fd) < 0)
    exit(1);
  if(fork() == 0){
    close(fd[1]);
    stage(fd[0]);
  }
  close(fd[0]);
  for(int i = 2; i <= LIMIT; i++)
    write(fd[1], &i, sizeof(i));
  close(fd[1]);
  wait(0);
  exit(0);
}
```

3.3 Pipeline Architecture Diagram

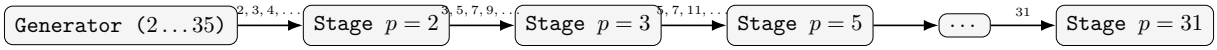


Figure 1: Streaming pipeline of concurrent filter processes connected by anonymous pipes.

3.4 Execution Trace

```
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
```

3.5 Answers to Questions (Observe – Part 2)

(1) How many processes exist at the moment "prime 31" is printed?

At the moment `"prime 31"` is printed, there are **12 processes** simultaneously alive in xv6:

1. The original **main generator process**, which generated the input sequence $2 \ldots 35$ and is currently waiting in `wait(0)` for its child to exit.
2. Plus **11 filter stage processes**, each managing one discovered prime:

$$p \in \{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31\}$$

Because each stage stays alive while candidate numbers flow through the pipeline, all 11 stages and the main generator exist concurrently:

$$1 \text{ (generator)} + 11 \text{ (prime filter stages)} = \mathbf{12} \text{ processes.}$$

(2) Change the limit from 35 to 280. Does it still work? If not, which limit did you hit (NOFILE per process, NPROC processes, or NFILE open files in the whole system) and how did you find out?

- **Outcome**: It does not work up to 280. It correctly prints 49 primes up to **227**, and then fails on the next allocation with the diagnostic error:

```
primes: pipe failed after prime 227
```

- **Limit Hit**: The system-wide file table limit, `NFILE = 100` (declared in `kernel/param.h`).
- **Why `NFILE` and not `NOFILE` or `NPROC`?**
  1. *Per-process file descriptors (`NOFILE = 16`)*: Every stage immediately closes `fd[0]` in the parent, `fd[1]` in the child, and the inherited input descriptors. Therefore, each process holds only 4 open file descriptors, well within `NOFILE = 16`.
  2. *Process table limit (`NPROC = 64`)*: 49 prime stages plus the generator equal 50 processes, which is below `NPROC = 64`.
  3. *System-wide file table (`NFILE = 100`)*: In xv6, every anonymous pipe requires **two** `struct file` instances in the kernel's global file table (`ftable` in `kernel/file.c`)—one for read and one for write. Active stages and initial console descriptors consume all 100 slots (49 pipes $\times$ 2 = 98 file slots + console/sh descriptors). When stage 227 tries to allocate the 50th pipe, `filealloc()` returns null and `pipe()` returns -1.
- **How we found out**: By checking the return value of `pipe(fd) < 0` in `primes.c`. The diagnostic printed `primes: pipe failed after prime 227`. Tracing `sys_pipe()` in `kernel/sysfile.c` reveals that `filealloc()` failed because all 100 `struct file` entries in `ftable` were exhausted.

(3) In two sentences: why is this a sieve that runs in parallel?

Each prime filter stage executes as an autonomous concurrent process that streams non-multiples into its output pipe as soon as candidate integers arrive, rather than waiting for prior stages to process the entire dataset. Multiple integers at different filtering depths are evaluated simultaneously across CPU cores, achieving streaming pipeline parallelism.

4 Part 3: Stretch Assignment – Implementing `xargs` (`xargs.c`)

4.1 Specification and Design

The `xargs` utility reads arguments from standard input and runs the specified command once per input line, appending the parsed arguments from the line:

- In xv6 there is no `$PATH` variable; `exec()` is invoked using the exact path provided.
- Space-separated arguments on a line are parsed into an argument vector `args[]`.
- For each line, `xargs` forks a child process, executes the command with `exec()`, and waits for completion.

4.2 Implementation of `user/xargs.c`

```c
#include "kernel/types.h"
#include "kernel/param.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  char buf[512], *args[MAXARG], *s;
  int n, r, k, i;
  char c;

  for(i = 1; i < argc && i < MAXARG - 1; i++)
    args[i-1] = argv[i];
  int base = i - 1;

  for(;;){
    n = 0;
    while((r = read(0, &c, 1)) == 1 && c != '\n'){
      if(n < sizeof(buf) - 1)
        buf[n++] = c;
    }
    if(r != 1 && n == 0)
      break;
    buf[n] = 0;

    k = base;
    s = buf;
    while(*s && k < MAXARG - 1){
      while(*s == ' ') *s++ = 0;
      if(*s == 0) break;
      args[k++] = s;
      while(*s && *s != ' ') s++;
    }
    args[k] = 0;

    if(fork() == 0){
      exec(args[0], args);
      fprintf(2, "xargs: exec %s failed\n", args[0]);
      exit(1);
```

```
    }
    wait(0);
    if(r != 1)
      break;
  }
  exit(0);
}
```

4.3 Execution Trace and Verification

We verified `xargs` inside xv6:

```
$ echo hello too | xargs echo bye
bye hello too
```

The words `hello` and `too` were parsed from standard input and appended as additional arguments to `echo bye`, outputting `bye hello too`.

5 Conclusions

Laboratory Work Nr. 8 provided deep, practical insights into low-level inter-process communication and concurrent systems programming in the xv6 kernel:

1. **Unidirectional IPC Mechanics**: The `pingpong` experiment demonstrated why bidirectional message exchange requires two separate pipes. Synchronization is governed naturally by the kernel sleep/wakeup mechanism inside `read()` and `write()`.

2. **Concurrent Pipeline Decomposition**: Implementing Doug McIlroy's prime sieve showcased the power of decomposing algorithmic problems into pipeline stages. Data flows concurrently through kernel ring buffers, providing pipeline parallelism across multiple processor cores.

3. **Operating System Resource Hierarchies**: The limit 280 test demonstrated how kernel table sizing defines architectural limits. While per-process descriptor limits (`NOFILE`) and process limits (`NPROC`) were satisfied, execution halted due to the global open file table limit (`NFILE = 100`).

4. **UNIX Pipeline Composability**: Building `xargs` completed the bridge between stream-oriented pipe outputs and argument-based program invocation, using standard `fork()`, `exec()`, and `wait()` semantics.